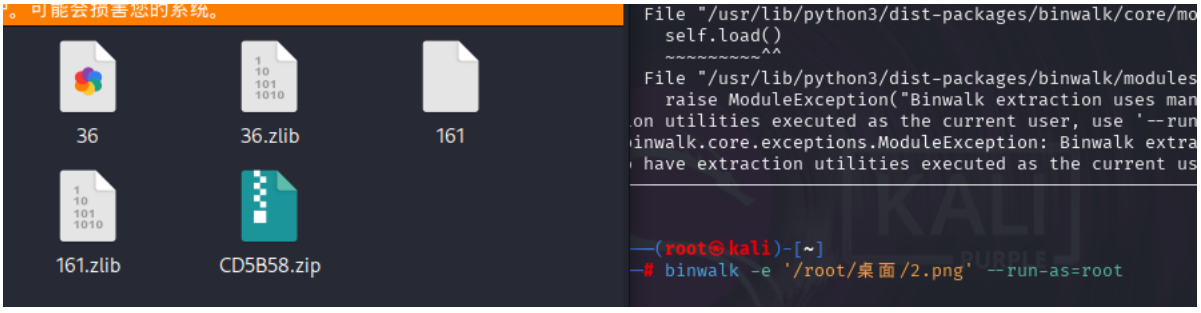
附件是一张照片 下载完之后 放进脚本里面 拿到一串类似于密码的东西

```
from PIL import Image
from tqdm import tqdm
img = Image.open("2.png").convert("RGB")
sumR, sumG, sumB = 0, 0, 0
for y in tqdm(range(img.height)):
    for x in range(img.width):
        r, g, b = img.getpixel((x, y))
        sumR += r
        sumG += g
        sumB += b

print(f"{sumR*0.6+sumG*0.3+sumB*0.1}") #1375729349.6
```

```
1375729349.6
```

我们用分离文件 发现里面有文件



发现有个加密的压缩包 密码就是 1375729349.6 解密后拿到.wav 文件 再跑下面脚本

```
import scipy.io.wavfile as wavfile
import numpy as np


def decode_non_standard_manchester(filename="2.wav",
                                   start_time_sec=6.0,
                                   segment_duration_sec=0.1):
    try:
        sample_rate, data = wavfile.read(filename)
    except FileNotFoundError:
        print(f"错误: 文件 '{filename}' 未找到。")
        return ""
    except Exception as e:
        print(f"读取WAV文件时发生错误: {e}")
        return ""

    if data.ndim == 2:
        audio_signal = data[:, 0]
    else:
        audio_signal = data

    start_sample = int(start_time_sec * sample_rate)
    samples_per_segment = int(segment_duration_sec * sample_rate)

    if start_sample + samples_per_segment > len(audio_signal):
```

```
        print(f"错误：开始时间 ({start_time_sec}s) 太靠后，或音频文件太短，无法处理至少一个分段。")
        return ""

    decoded_bits = []
    current_pos_sample = start_sample

    threshold = 0

    print(f"采样率: {sample_rate} Hz")
    print(f"每个分段的采样点数: {samples_per_segment}")
    print(f"从采样点 {start_sample} 开始处理")
    segment_count = 0

    while current_pos_sample + samples_per_segment <= len(audio_signal):
        segment_data = audio_signal[current_pos_sample: current_pos_sample + samples_per_segment]

        binary_segment = (segment_data > threshold).astype(int)

        if np.all(binary_segment == 1):
            decoded_bits.append('0')
        elif np.any(np.diff(binary_segment) == -1):
            decoded_bits.append('1')

        current_pos_sample += samples_per_segment
        segment_count += 1

    print(f"共处理了 {segment_count} 个分段。")
    return "".join(decoded_bits)


if __name__ == "__main__":
    decoded_sequence = decode_non_standard_manchester()
    if decoded_sequence:
        print("\n解码后的序列:")
        print(decoded_sequence)
```

解码后的序列:
0101011101101000011010010111001101110000011001010111001 0

拿到序列后 放进随波逐流

[随波逐流]CTF编码工具 V6.7 20250418　最新版本为：V6.7 20250503

Base/Rot　字密1　字密2　字密3　字密4　编码转换　带key解密　多key解密　在线解密　进制转换　其他工具　文件　图片　题库&更新

密文↓(字:56)　密钥key/str/url:　一键解码

```
)101011101101000011010010111001101110000011001010111001 0
```

解密结果↓　正则搜　搜索　↑结果↑

```
一 键 解 码:      |解码结果 (注：在线解密密码不参与一键解码)
base64解码:       S]5S]uS]tWM4S]tWM5S]uSMuS]uSM4S]tS]5S]uSMt
base32解码:
base16解码:       □□□□□□□□ □□□□□□□ □□□□ □□□□□□□ □
base85(a)解码:
base85(b)解码:
base58解码:
base36解码:       cazdc063owu19j7m15kh2hzim6nzb25qaka
base91解码:
base92解码:       (IBJ□VB¬e□(IXÊ&P¡□d(IY
□P¡%    (IY
□V
base62解码:       615551264659623667867364978375109224250642278937326667521141806813778681006628571622148157272919 8
base62(ASCII)解码:
base100解码:      °úù'ù°úúú'ù°úù
base45解码:
-----------------------------------------------------------
Base混合多重解码:
[解码1次] Base16
混合解码结果:□□□□□□□□ □□□□□□□ □□□□ □□□□□□□ □
-----------------------------------------------------------
16进制转字符:     □□□□□□□□ □□□□□□□ □□□□ □□□□□□□ □
10进制转字符:
8进制转字符:      □AHW@□A□ @HI □@A @□
2进制转字符:      英文: Whisper
中文: 圪楳蠱
混合进制解码:
培根bacon解码:    vclzndeptkg
```

本软件为测试之用，不得用于任何非法及商业用途 #　　随波逐流出品 www.1o1o.xyz